



A^3 : Adaptive Autonomous Allocation of TSCH Slots

Seohyang Kim

Samsung Research
Samsung Electronics
seohyang.kim@samsung.com

Hyung-Sin Kim

Graduate School of Data Science
Seoul National University
hyungkim@snu.ac.kr

Chong-kwon Kim

Department of Computer Science
Seoul National University
ckim@snu.ac.kr

ABSTRACT

Time Slotted Channel Hopping (TSCH), defined in the IEEE 802.15.4e standard requires complicated slot scheduling to enjoy its collision-free multihop communication capabilities fully. Recently, several autonomous scheduling algorithms that allocate slots without central control and additional control message exchange have been devised. However, autonomous algorithms are traffic oblivious, assigning the same number of slots regardless of traffic demands. Undifferentiated resource allotment is identified as the root cause that impairs the performance of TSCH networks. To address the issue, we propose A^3 , an autonomous and adaptive slot allocation scheme that adjusts the number of slots per slot-frame according to varying traffic loads. A key component of A^3 is *receiver-side load estimation* in real-time without any explicit control message exchange. A^3 can be combined with any autonomous scheduling protocols, such as Orchestra and ALICE. With extensive evaluation on a large public testbed comprising 62 nodes, we verify that A^3 significantly enhances autonomous scheduling algorithms' performance. It improves throughput by more than twice, PDR (Packet Delivery Ratio) by more than six times, and reduces the latency by ten times.

CCS CONCEPTS

• **Networks** → **Link-layer protocols**; *Cross-layer protocols*; Network performance evaluation.

KEYWORDS

802.15.4e, Time Slotted Channel Hopping, autonomous scheduling, multi-hop networks

ACM Reference Format:

Seohyang Kim, Hyung-Sin Kim, and Chong-kwon Kim. 2021. A^3 : Adaptive Autonomous Allocation of TSCH Slots. In *The 20th International Conference on Information Processing in Sensor Networks (co-located with CPS-IoT Week 2021) (IPSN '21)*, May 18–21, 2021, Nashville, TN, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3412382.3458273>

1 INTRODUCTION

Multihop wireless networks, which extend the periphery of networks with wireless relays, have been identified as a game-changer that shapes new network landscapes of the Internet of Things (IoT). In addition to apparent benefits such as energy frugality and easy deployment, multihop wireless networks provide hyper-connection capabilities. They enable flexible, impromptu connections to a massive number of tiny devices with limited resources. However, multihop networks also suffer from the problem of severe interference. As pointed out in [14], the performance of multihop wireless networks drops sharply as the number of wireless hops increases. Even though multihop networks may suit generous performance applications such as telemetry, they can hardly be a viable option for mission-critical applications.

To satisfy the reliability and high-performance communication requirements of mission-critical applications while enjoying the benefits of multihop connectivity, IEEE defined the Time Slotted Channel Hopping (TSCH) protocol in 802.15.4e standard [30]. Here, to avoid interference and collisions due to uncoordinated transmissions, nodes are time-synchronized and communicate using multiple channels while hopping to different channels continuously. The communication between a node pair is executed at a coordinated slot and channel. Although TSCH avoids collisions and interference through frequency diversity, and energy wastage through time synchronization [12][3], a well-designed slot (communication resource) scheduler is necessary for the thorough realization of its potential. The standard excludes the specification on schedule, leaving room for research on innovative scheduling protocols.

Despite decades of research on adaptive resource scheduling for wireless networks, TSCH slot scheduling still has unique challenges in that it is for LLNs (Low-Power Lossy Networks) consisting of *battery-powered, resource-constrained*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IPSN '21, May 18–21, 2021, Nashville, TN, USA

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8098-0/21/05...\$15.00

<https://doi.org/10.1145/3412382.3458273>

nodes operated in interference-prone ISM bands. In addition to the fundamental requirement, real-time allocation of just-enough resource to cope with time-varying traffic load, TSCH scheduler must satisfy the following requirements to have a practical value in multihop LLNs: (1) robust operation on dynamic routing topology, (2) collision resolution, (3) simple distributed operation, (4) low control packet overhead, and (5) low computation overhead [15]. Although there have been a number of adaptive slot schedulers for TSCH, operating in a centralized or distributed manner [26][31][1][32][27][35], they fail to satisfy these requirements and have never been demonstrated successfully in a real-world testbed.

To make a breakthrough, *autonomous* schedulers have been proposed recently, such as Orchestra [6] and ALICE [21]. The prime virtue of autonomous scheduling is eradicating control information exchanges and central controls required for distributed or centralized scheduling. These autonomous schemes tightly interact with the routing layer and utilize a simple hash function, enabling each node to find timeslots to communicate with its routing neighbors autonomously without additional control overhead. They also provide a slotframe architecture for reliable delivery of routing control packets and seamless scheduling service on routing topology changes. However, on the flip side, previously devised autonomous schedulers have sacrificed an essential aspect for simplicity; all links in the whole network use the “same and fixed” number of slots regardless of traffic load. Given that each link experiences significantly different and time-varying traffic load depending on various factors, such as node location, node density, dynamic routing topology, and application behavior [15], the inflexible approaches cause either packet losses (under-provision) or energy wastage (over-provision), or both.

Challenges: The next step is clear: equipping autonomous schedulers with traffic-aware adaptive scheduling capability while keeping all of their positive characteristics. To this end, a sender and a receiver of each directional link must fulfill the following three requirements: (1) the sender and the receiver should estimate time-varying traffic load for the directional link *autonomously* without control packet exchanges. While it is trivial for the sender to know its outgoing traffic load to the receiver, the challenge lies in the receiver estimating its incoming traffic load from the sender. (2) Given the estimated traffic load information, the two nodes should adjust their slot allocation *autonomously*. (3) Although there is no direct message exchange, the slot allocation for the node pair should be well *synchronized* in terms of both time and the number of slots.

Approach: We design and implement a fully *autonomous adaptive slot allocation algorithm* called A^3 . The algorithm is

protocol independent in that it can be integrated with any autonomous schedulers, including Orchestra and ALICE. In A^3 , a receiver estimates the corresponding sender’s transmission attempt rate in real-time and adjusts the number of slots per slotframe dynamically based on the estimated attempt rates. Among many estimation statistics, we employ the EWMA (Exponentially Weighted Moving Average) scheme because it provides fast detection as well as stability. We perform a mathematical analysis of the EWMA scheme to show the impact of its parameters on performance.

We evaluate A^3 on Orchestra and ALICE, verifying that it detects changes in traffic load quickly. Senders and receivers are well synchronized with occasional and noncritical mismatches. A^3 improves the legacy autonomous schedulers’ performance by doubling throughput, increasing PDR by six times, and reducing latency by ten times.

Contributions: Our contributions are as follows:

- We propose A^3 including a traffic load estimation scheme that can be applied in any autonomous scheduler as an *independent* module.
- We design the scheme to operate in an *autonomous* manner: a receiver self-estimates the traffic load at the sender based on the implicit local information without any control packet exchange.
- We implement A^3 in the most widely used low-power operating system, Contiki, and open the source code. The implementation is on top of two state-of-the-art autonomous schedulers: Orchestra and ALICE.
- We verify the performance of A^3 in a large public testbed [2] comprised of 62 nodes, showing up to 6 times improvement in PDR and ten times reduction in latency compared to state of the art.

2 BACKGROUND

Time Slotted Channel Hopping (TSCH) is a MAC layer protocol defined in the IEEE 802.15.4e standard [30]. The standard is the enhanced version of 802.15.4 [29] applying *time synchronization* and *channel hopping* techniques for the purpose of reliable and efficient communications. For time synchronization, each node has its own time source node and synchronizes its time to the time source node. Each time slot is identified by Absolute Slot Number (ASN) which increases by one at the end of each time slot.

The following is a brief introduction to how multiple nodes build a multihop TSCH network. At first, a node that is set as a TSCH PAN coordinator creates its own time-slotted network by setting ASN of the first time slot as 0. ASN of this TSCH network increases by one at the end of each time slot, and the PAN coordinator broadcasts enhanced beacon (EB) periodically. EB includes various information such as the current ASN, PAN ID and Frequency Hopping Sequence

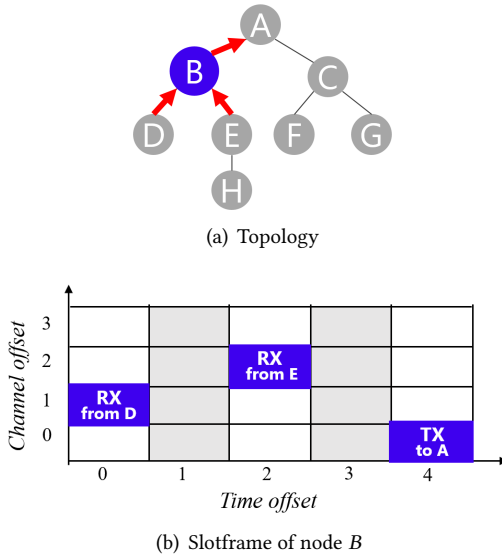


Figure 1: Example of TSCH slotframe schedule

(FHS), etc. Any node that has received the EB joins the TSCH network using the EB's information and sets the EB sender as its time source node. This node also broadcasts EB periodically, and any node that has received the EB from this node sets this node as its time source node. By repeating this process, multiple nodes build a time-synchronized multihop network.

The most challenging problem in applying TSCH is a *slot scheduling*. There are a lot of nodes in a multihop network, and to make each node pair exchange packets successfully, we should carefully allocate a time slot and channel for communication between a node pair. To simplify the schedule, IEEE 802.15.4e introduces a two-dimensional time-frequency schedule table called *slotframe*. Figure 1 shows an example of routing topology and the slotframe of node B. In this example, there are 5 time offsets (x axis) and 4 channel offsets (y axis), creating 20 slots. The slotframe length (L_{SF}), i.e., the number of timeslots in a slotframe, is 5. Using a shared ASN, L_{SF} and modulo operation, a node calculates the current time offset that is $ASN \% L_{SF}$. Channel offset permits frequency-domain multiplexing. A node calculates the channel of the current time slot by using frequency hopping sequence, ASN, and channel offset as follows: $FHS[(ASN + channelOffset) \% L_{FHS}]$ where FHS and L_{FHS} are the frequency hopping sequence and its length. Note that a channel offset does not mean the used channel.

Let's see the scheduling example of node B (Figure 1(b)). Following the schedule table, it receives a packet from node D at slot (0,1). So, it should wake up and listen at slot (0,1) to receive a packet from node D. If node D does not have any packet to send to node B, it will not transmit any packet. Regardless of this, node B does not know whether its neighbor

has a packet to send or not, so it should always listen at the scheduled slot. Similarly, node B is scheduled to receive a packet from node E at slot (2,2) where it should listen to the channel to receive a packet from node E. At slot (4,0), node B is scheduled to transmit a packet to node A, meaning that node A listens to the corresponding channel at the slot (4,0) to receive a packet from node B successfully.

The quality of the slotframe schedule directly affects the performance of TSCH. With a well-designed slotframe schedule, we can efficiently utilize the given resource (time and channel) to efficiently achieve high reliability (packet delivery ratio), low end-to-end latency, and low radio duty cycle.

3 RELATED WORK

Given that there have been numerous traffic-sensitive scheduling techniques such as proportional fair scheduling in powerful cellular systems, those for TSCH have also been developed, which are categorized into centralized and decentralized (distributed) schemes.

3.1 Adaptive Centralized Schedulers

In centralized schedulers, a central node (e.g. TSCH PAN coordinator) schedules the schedule table of all the nodes in a TSCH network. To this end, the central node collects all the network information from the participating nodes and fabricates the global view on a network. It disposes of the network-wide slotframe schedule and distributes the slotframe assignment to each node. Since it has significant scheduling overheads generating a lot of control packets and latency for the schedule, this method is hard to apply to topology volatile environments and generally recommended to be used on fixed network topology. TASA [26] and MOD-ESA [31] are the representative centralized schedulers. They make a central node schedule slotframes for all nodes according to global information on traffic load and conflict relation among nodes. LLTT [33] also builds a fixed TSCH network to reduce end-to-end latency. However, these schemes assume fixed routing topology and constant traffic load, which does not hold in the real world.

3.2 Adaptive Distributed Schedulers

In distributed schedulers, the scheduling function is distributed over the nodes. Although distributed schedulers diminish the overhead of control packets compared to centralized schedulers, they still suffer from additional traffic overhead and information mismatch, especially on dynamic networks. For example, DeTAS [1] and Wave [32] let each node allocate time slots adapting to local traffic demands, and Wave also utilizes interference relation between nodes. However, each node's schedule highly depends on its parent's schedule, and only the root node can initiate reschedule.

OTF [27] proposed an on-the-fly slot allocation method to overcoming the inflexibility. Here, any node can initiate reschedule events and update its slotframe through the negotiation process with its neighbors. Recently, the IETF 6TiSCH working group [36] standardized 6P [35] protocol that enables negotiation between a node pair. Here, a node sends a request message announcing the available slot list, and the receiver responds to the sender by sending the list of chosen slots. Similarly, D-MSR [37] and MPLS [24] also use a handshaking process between a node pair for adequate bandwidth reservation. Due to the negotiation processes, all distributed algorithms cannot quickly adapt to the dynamically changing routing topology showing long adaptation latency and a large volume of control traffic overhead.

3.3 Fixed Autonomous Schedulers

Unfortunately, none of the adaptive scheduling techniques have been demonstrated on large-scale real-world testbeds. Recall that a successful low-power network protocol requires a novel idea and a careful design and implementation to handle various critical issues, such as low control overhead and robust operation with time-varying routing topology. To this end, two autonomous schedulers have been proposed, Orchestra [6] and ALICE [21], which sacrifice adaptive scheduling capability for lightweight and robust operation in a multihop network that allows dynamic routing topology changes.

Specifically, the autonomous schedulers' key idea is to enable each node to resolve its slotframe autonomously by hashing a node (or directional link) ID rather than exchanging control packets. The hash function resolves timeslot collision in a pseudo-random manner. To support unicast communication with a neighbor, these schemes access the routing layer to get the routing neighbor information and calculate a unique timeslot position based on the neighbor's ID. When the routing topology is changed, the new neighbor information at the routing layer triggers an autonomous and immediate update in the schedule table (slotframe). Orchestra and ALICE have been demonstrated on large public testbeds and provide open-source codes, regarded as state of the art in the TSCH scheduling regime [7].

3.4 Towards Adaptive Autonomous Schedulers

Autonomous schedulers, lacking one crucial feature - adaptability, are not consummate solutions. Without adaptability, they fail to meet fluctuating traffic demands, which change according to various factors, such as node location, node density, routing topology, and application behavior. To support various modern applications in a multihop network, adaptive scheduling capability must be added. The recent studies

in [16] and [15] also have proposed the traffic-aware distributed scheduling method. However, they do not preserve the autonomy feature because they rely on the exchanged control information.

In this work, we propose *autonomous* and *adaptive* TSCH slot allocation algorithm, called A^3 . A^3 is protocol-independent and can be used together with any autonomous scheduler such as Orchestra or ALICE. A^3 autonomously estimates each neighbor's traffic load in real-time and adapts the number of receive and transmission slots.

4 PRELIMINARY STUDY

In this section, we investigate the two most widely used autonomous schedulers, Orchestra [6], and ALICE [21]. Both protocols use hashing for slot position determination; Orchestra hashes node ID while ALICE hashes directional link ID. ALICE further incorporates various techniques to improve performance, such as time-varying scheduling, on-the-fly packet selection, and multiple channel offset usage. We add the three functions to Orchestra and call the modified version eOrchestra (enhanced Orchestra). This section evaluates the performance of eOrchestra and ALICE. For evaluation, we use RPL [34], the IPv6 standard routing protocol for LLNs, for the routing protocol, and 62 M3 nodes in FIT IoT-LAB [2] public testbed located in Grenoble, France. We use Orchestra sender-based mode since it performs better than the receiver-based mode [21].

4.1 Impact of Unicast Slotframe Length

We first evaluate the performance according to the unicast slotframe length. Each scheme allocates the same number of slots per frame at a given routing topology regardless of the slotframe length. Thereby, communication opportunity increases as the slotframe length decreases. Each node sends and receives four packets per minute to and from the root, respectively. Note that although the average traffic load is 4 pkts/min (i.e., 15 seconds/pkt), packet interval is not constant but set randomly at each time, between 0 seconds and 30 seconds.

Figure 2 shows the results. When the slotframe length is short, both schemes provide a high packet delivery ratio (PDR) and low latency at the expense of the radio duty cycle (Figures 2(a)-(c)). As the slotframe length increases, the radio duty cycle is improved, but PDR drastically decreases, and the latency increases due to a lack of communication opportunities. While ALICE outperforms Orchestra, the performance of both schemes is significantly degraded with long slotframes. This is because both schemes provide *the fixed number* of tx/rx slots per slotframe *regardless* of traffic load. Note that when a node has N neighbors, Orchestra allocates

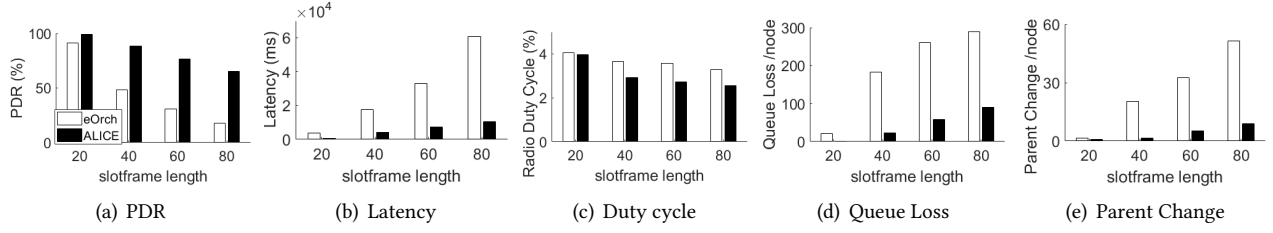


Figure 2: Performance by the slotframe length in a dynamic topology

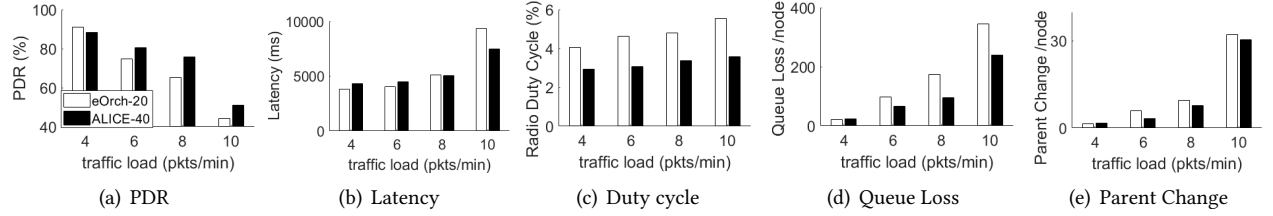


Figure 3: Performance by the traffic load in a dynamic topology

one tx slot and N rx slots in a slotframe, and ALICE allocates N tx slots and N rx slots in a slotframe. Due to insufficient communication opportunities at long slotframe lengths, packets suffer long queueing delays and/or are dropped from the transmission queue (Figure 2(d)). Consequently, the network becomes unstable, resulting in continuous RPL parent changes (Figure 2(e)).

4.2 Impact of Traffic Load

Next, we evaluate the impact of traffic load when a slotframe length is fixed. Since ALICE provides better performance at the same slotframe length, we use a longer slotframe for ALICE for a fair comparison. Thereby, we assign 40 for ALICE and 20 for eOrchestra.

Figure 3 shows the performance according to traffic load. A traffic load of k pkts/min means that each node sends k packets per minute (i.e., pkts/min) to the root and receives k pkts/min from the root on average. Since eOrchestra uses a smaller slotframe length compared to ALICE, it requires a higher radio duty cycle (Figure 3(c)). Regardless of the assigned traffic load, both schemes allocate the *fixed* number of tx/rx slots in a slotframe based on the number of neighbors. Consequently, as the traffic load increases, the transmission queue becomes full, increasing queueing delay and latency (Figure 3(b)). Further increases in traffic demands over the network capacity causes packet drops (Figure 3(d)). Packet losses make the routing topology unstable (Figure 3(e)), resulting in more control packets as an effort to fix the situation. However, the control packets cannot be delivered due to congestion, which causes a vicious cycle. Therefore, PDR of both

schemes drastically decreases (Figure 3(a)) as the traffic load increases.

4.3 Problem Definition

It is obvious that the current autonomous schedulers have limitations in their performance. The fundamental problem is that they do not provide *adaptability*. In their methods, each node just considers the number of its neighbors regardless of the actual traffic load. However, in multihop networks, nodes experience different traffic loads that can vary dynamically over time.

A naive solution would be considering not only the number of (one-hop) neighbors but also the relative location from the root or the size of the sub-tree nodes. It partially makes sense because traffic load from/to a node is likely to be proportional to the number of its sub-tree nodes instead of the number of neighbors. However, when each node generates a different amount of traffic, a node's sub-tree size cannot represent its actual traffic load. A node with a large sub-tree can experience an extremely low traffic load and does not need many tx/rx slots. In addition, since traffic loads and wireless channel conditions fluctuate dynamically, any static scheme can lose their efficiency and effectiveness when faced with dynamic changes.

Therefore, rather than depending only on topological information, we should directly use the real-time traffic load information for resource allocation. Simultaneously, we should preserve the *autonomous* feature, which makes autonomous scheduling most widely and commonly used these days.

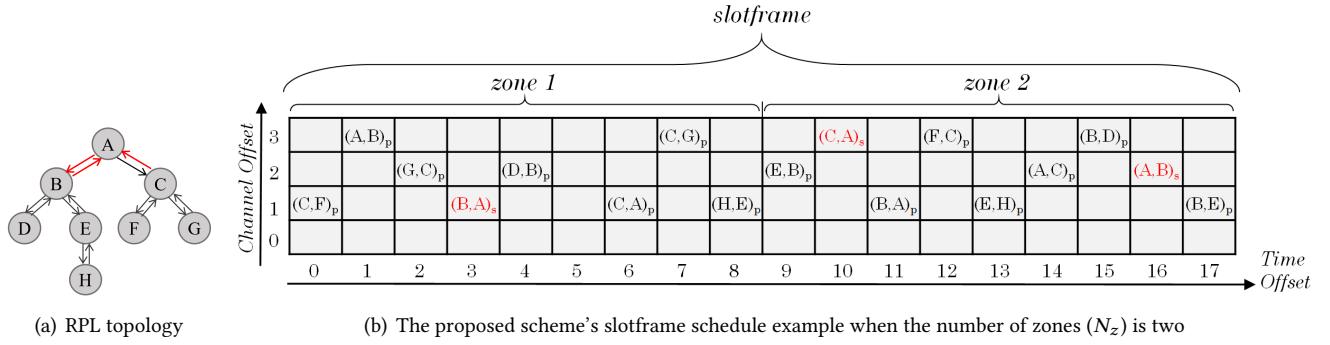


Figure 4: An example of scheduling using the proposed scheme

5 A^3 DESIGN

We propose an autonomous and adaptive slot allocation scheme called A^3 where each node adjusts its tx/rx slots according to time-varying traffic load to/from its neighbors. A^3 's key component is its *load estimation algorithm* that, at receiver nodes, fathoms the traffic loads from the remote sender nodes in real-time. A^3 operates in a fully autonomous manner and is independent of underlying schedulers such that it can be applied to ALICE as well as Orchestra.

5.1 Overview

A^3 has two components: *load estimation* and *multi-slot scheduling*. Basically, a scheduler with the proposed adaptation module arranges the slotframe by following its own scheduling rules. It assigns additional slots only when they are needed and removes the slots immediately when they are no longer needed. Thereby, nodes with high traffic load utilize more slots, not dropping packets, while nodes with low traffic load occupy a small number of slots saving both resources and energy. The traffic load on a node varies over time depending on the application behaviors and routing topology. The proposed method A^3 continuously monitors the real-time traffic loads and adjusts the number of tx and rx slots needed.

To describe our mechanism, consider a directional link comprising a sender and a receiver, which is the basic communication unit. On the sender side, traffic load determination is relatively easy because the sender can monitor the transmission queue for the receiver. If the traffic load toward the receiver is larger/smaller than a pre-determined increase/decrease threshold, the sender increases/decreases the number of tx slots in a slotframe for the receiver. On the other hand, to keep the autonomy feature, the receiver must *self-estimate* the corresponding sender's transmission attempt rate (incoming traffic load) and increases/decreases

its rx slots for the sender based on its own *different* thresholds. The disparity between the two estimations is critical to performance and will be treated in detail later.

After determining the number of tx/rx slots for a directional link, the tx/rx slots' locations on a slotframe are scheduled. To this end, a slotframe is divided into N_z zones (Figure 4(b)), and each directional link selects one of them as its own primary zone. One default slot is scheduled on the primary zone, and the additionally allocated slots are scheduled on the other non-primary zones. Note that a node can use one to N_z slots per slotframe, and allocate up to one slot per zone. When determining the primary zone's location in a slotframe and a tx/rx slot's location in each zone, a hash function is fed with the sender and receiver node IDs as the input keys. The use of the hash function avoids collision among tx/rx slots of multiple directional links in a pseudo-random manner. We additionally use time information ($ASFN^1$) as the input to the hash function so that each directional link's tx/rx slot locations change every slotframe. It prevents tx/rx slots of specific unfortunate links from colliding repetitively.

Suppose the two estimations of the traffic load are the same. In that case, the time-varying numbers and locations of tx and rx slots of a sender-receiver pair match each other even though the sender and the receiver do not exchange any control packet. Besides, collision among multiple links' tx/rx slots is avoided without explicit coordination.

Figure 4(b) shows a scheduling example of the proposed module on an 8-node RPL routing topology (Figure 4(a)). Nodes A, B, and C are on the root-side of the routing tree with more traffic loads than the other nodes, requiring more tx/rx slots. A slotframe is divided into $N_z = 2$ zones, thereby A^3 allows each directional link to have up to two slots in a slotframe. Each link (X, Y) has its primary zone, and its

¹ Absolute SlotFrame Number (ASFN) is a known value for the time synchronization in TSCH. This value indicates how many times the slotframe has repeated from the start of the current TSCH network ($ASFN = \lfloor ASN / L_{SF} \rfloor$ where $\lfloor x \rfloor$ is the floor function).

default (primary) slot denoted as $(X, Y)_p$ is scheduled on its primary zone. Nodes A , B and C schedule additional (secondary) slots denoted as $((A, B)_s, (B, A)_s$ and $(C, A)_s$) to support heavier traffic loads. These extra slots are scheduled on non-primary zones. Note that each link has a different primary zone and different tx/rx slot locations due to the hash function. Moreover, since the hash function also uses time information as a part of the input key, we do not have to worry about occasional crowded zones or slots at a specific time because the schedule will be changed at the next ASFN delivering the long-term fairness.

5.2 Traffic Load Estimation

Consider a directional link (A, B) with A as a sender and B as a receiver. The sender can easily know the directional link's traffic load (denoted as a) by monitoring its own tx slots' transmission attempts. To this end, among many estimation statistics such as CUSUM (CUMulative SUM) and SMA (Simple Moving Average), we use EWMA (Exponentially Weighted Moving Average) because it provides quick detection as well as stability. Letting $\hat{a}_{i,tx}$ be the sender's estimated transmission attempt rate at the i^{th} tx slot. The sender updates $\hat{a}_{i,tx}$ as follows.

$$\hat{a}_{i,tx} = \begin{cases} (1 - \lambda) \cdot \hat{a}_{i-1,tx} & \text{No Tx} \\ (1 - \lambda) \cdot \hat{a}_{i-1,tx} + \lambda & \text{Tx} \end{cases} \quad (1)$$

λ ($0 < \lambda \leq 1$) is a weight for a new sample. A large λ yields fast decay of old information and thus leads to fast detection at the cost of low stability. Our mathematical analysis suggests setting λ to ~ 0.1 , and we choose λ from an interval of $[0.05, 0.13]$ depending on observation frequencies. The frequency of sampling is inversely proportional to the length of slotframe. We use small and large λ for short and long slotframes, respectively. We set the initial value $\hat{a}_0$ to 0.5.

The more challenging task of A³'s traffic load estimation module is to make the receiver estimate the sender's transmission attempt rate (denoted as $\hat{a}_{i,rx}$) correctly and *autonomously* without any explicit information exchange. To this end, the receiver gets the implicit local information by sampling each rx slot's channel status and classifying it as the five types: Idle, Others, Success, Collision, and Unavailable.

(1) Idle: No transmission energy was detected. The receiver regards that the sender did not transmit a packet in this slot and updates $\hat{a}_{i,rx}$ accordingly.

(2) Others: The receiver overheard a packet not relevant to the corresponding link. It regards that the sender was idle in this slot and updates $\hat{a}_{i,rx}$ accordingly.

(3) Success: The receiver successfully received a packet sent from the corresponding sender. It regards that the sender

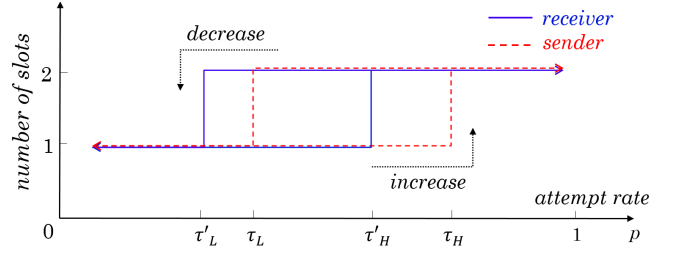


Figure 5: Increase and Decrease thresholds for sender and receiver

transmitted a packet in this slot and updates $\hat{a}_{i,rx}$ accordingly.

(4) Collision: The receiver sensed transmission energy but could not parse a packet due to some reasons such as collision. In this case, the sender might or might not have transmitted a packet. Therefore the receiver does not use this sample (no update).

(5) Unavailable: The receiver was not able to listen to the channel at this slot due to some reasons (e.g., its own transmission²). So the receiver does not use this sample (no update).

We use EWMA with the same parameters for the receiver-side load estimation. Letting $\hat{a}_{i,rx}$ be the estimated transmission attempt rate at the i^{th} rx slot, the update rule of $\hat{a}_{i,rx}$ is as follows.

$$\hat{a}_{i,rx} = \begin{cases} (1 - \lambda) \cdot \hat{a}_{i-1,rx} & \text{No Tx (Idle or Others)} \\ (1 - \lambda) \cdot \hat{a}_{i-1,rx} + \lambda & \text{Tx (Success)} \\ \hat{a}_{i-1,rx} & \text{Collision or Unavailable} \end{cases} \quad (2)$$

5.3 Resource Allocation

Both senders and receivers determine the numbers of slots per slotframe based on directly observed or estimated transmission attempt rates, respectively. We employ the hysteresis for the stability of control. Figure 5 shows the two hysteresis cycles. A sender and a receiver increase/decrease the number of slots per slotframe from 1 to 2 or from 2 to 1. A sender and a receiver use different increase thresholds τ_H and τ'_H , respectively. Likewise, a sender and a receiver use different decrease thresholds, τ_L and τ'_L , respectively. When the observed/estimated transmission attempt is larger than the increase threshold τ_H/τ'_H , a sender/receiver doubles the corresponding link's slots. When the transmission attempt is lower than the decrease threshold τ_L/τ'_L , a sender/receiver reduces the number of slots by half.

²In TSCH, when multiple operations are scheduled at the same time slot, a transmission operation has a higher priority than a listen operation.

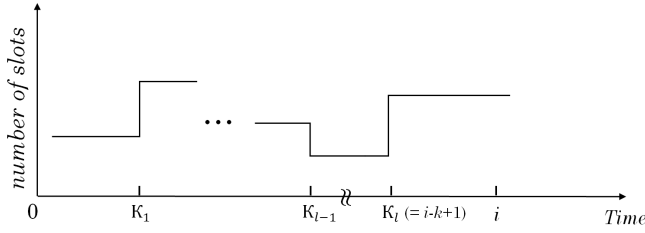


Figure 6: Changes in attempt rate and EWMA

We decide to set $\tau_H \geq \tau'_H$ and $\tau_L \geq \tau'_L$. This means that when traffic load increases, the receiver may increase its rx slots earlier than the sender increases its tx slots. On the other hand, when traffic load decreases, the receiver may decrease its rx slots later than the sender decreases its tx slots. This is because, when the sender allocates more tx slots than the receiver's rx slots, it will cause sterile transmission interfering with the other node pairs' communication and deteriorating the network performance.

5.4 Parameter Selection

The threshold parameters that determine the increase and decrease of the number of allocated slots can critically impact the proposed scheme's performance. Also, the EWMA weight, λ , controls the decay speed of old samples. Here, we perform a mathematical analysis that will be used as a guide in selecting proper parameter values.

Consider a directional link of a sender and a receiver. Assume that the current observation period is i and the changes in the number of slots per slotframe occur at $\kappa_1, \kappa_2, \dots, \kappa_l$ as shown in Figure 6.³

Let us assume that κ_l is the last time where the number of slots per slotframe for the directional link was changed to a different value. Let us assume that the estimator observes k samples after the last change in the number of slots per slotframe (at κ_l). Then, the relation between κ_l and the current observation period i is $\kappa_l = i - k + 1$. Further, assume that the changes in the number of slots per slotframe occur slowly such that $\kappa_j - \kappa_{j-1} \gg 0$. Because transmission attempts are binary events, we model the transmission process as a *Bernoulli process* with a parameter p . Note that when the attempt rate changes, it is reflected in changes in p . Let p_c and p_o be the current attempt rate and the prior attempt rate, respectively. Lastly, note that the sender knows any change in p immediately but the receiver can only estimate it by using $E[Z_i]$.⁴

³The transmission attempt rate of a link is updated at every tx/rx slot while the number of slots per slotframe is updated only when the transmission attempt rate is larger/smaller than the increase/decrease thresholds.

⁴ $E[Z_i]$ is the expected value of the transmission attempt rate of a sender that the receiver estimates.

Because the slot increase process and slot decrease process are mathematically similar, we focus on the slot increase case. Assume that, at $\kappa_l = i - k + 1$, the actual attempt rate p increases to a value greater than τ_H where τ_H is the sender's increase threshold. While the sender recognizes the increases in the attempt rate immediately and doubles the number of allocated slots, the receiver requires several samples for statistically reliable recognition via $E[Z_i]$. Because the receiver detects shifts slower than the sender and because $E[Z_i]$ is less than or equal to p_c in increasing phases, the receiver should use a threshold $\tau'_H \leq \tau_H$. Let $\Delta_H = \tau_H - \tau'_H$. A larger gap (large Δ_H) reduces the probability of false-negative at the cost of large false-positive probabilities. The false-positive case might only result in additional energy consumption due to idle listening. However, the false-negative case might cause invalid transmission attempts (the sender transmits while the receiver does not listen to the channel), which in turn degrades overall network performance. So we should design the module to reduce the false-negative case rather than the false-positive case.

Even though the exact computation of false-negative probability is not amenable, we derive its upper bound and use the bound to choose proper threshold values.

Lemma 1: *The false-negative probability is upper bounded by*

$$\frac{\lambda^2}{\lambda^2 + (4(2 - \lambda)(\Delta_H - (1 - \lambda)^k(\tau_H - p_o)))^2} \quad (3)$$

at $p_c = \tau_H$.

Proof. The detailed proof can be found in **Appendix A**.

First, we analyze the impact of weight λ on the detection speed and the false-negative probability. The expectation of an EWMA statistic Z_i is $(1 - (1 - \lambda)^k) \cdot p_c + (1 - \lambda)^k \cdot p_o$. Its variance is bounded by $\frac{\lambda}{4 \cdot (2 - \lambda)}$. A larger λ leads to fast convergence of expectation to p_c that again results in fast detection (detection with a small number of samples, k). However, a larger λ increases the variance and inflates the false-negative probability. A large λ also hurts the stability of detection schemes. Because samples are obtained every slotframe time, we determined to use λ around 0.10. At $\lambda = 0.1$, the 90% of rate changes are reflected in the estimator after 22 samples.

Now, suppose that we fix λ and k to 0.1 and 22, respectively, such that 90% of shift is reflected. Let α be the target false-negative probability. The upper bound reveals that Δ_H is not sensitive to the target false-negative probability. At $\tau_H - p_o = 0.2$, Δ_H increases from 0.032 to 0.033 only as α decreases from 0.05 to 0.01. On the contrary, Δ_H is sensitive to the EWMA weight parameter λ ; at $\alpha = 0.05$, $\tau_H - p_o = 0.2$, Δ_H increases from 0.032 to 0.093, as λ decreases from 0.10 to 0.05. Based on the observations, we choose to use 0.1 for Δ_H . By setting the

maximum (target) slot utilization as 0.75, we set $\tau_H = 0.75$ and $\tau'_H = 0.65$.

We also derive an upper bound of false-positive probability. False-positives are defined as the decisions to increase slots when the current attempt rate is less than equal to the increase threshold τ_H . We use $\tau_L = 0.34$ and $\tau'_L = 0.29$.

Lemma 2: *The false-positive probability is upper bounded by*

$$\frac{\lambda^2}{\lambda^2 + (4(2 - \lambda)(\tau'_H - (1 - \lambda)^k(p_c - p_o)))^2}. \quad (4)$$

Proof. The detailed proof can be found in **Appendix B**.

5.5 Multi-Slot Scheduling

After the traffic load estimation module decides the number of slots for the next slotframe, the scheduling module assigns the slots on a slotframe. In doing so, A³ tries to minimize slot collision among directional links.

Each slot is scheduled on a different zone in a slotframe. We use different slot ID (s_{id}) for each slot scheduled for a directional link. s_{id} of the primary slot is always 0 and additionally allocated slots have different s_{id} in increasing order. The primary zone of a directional link (k, l) at the $ASFN^{th}$ slotframe is decided as follows:

$$z_{pr}(k, l, ASFN) = Hash(\alpha ID(k) + ID(l) + ASFN, N_z). \quad (5)$$

The coefficient α is used to differentiate the direction of the link between the sender k and the receiver l . Thus, $\alpha ID(k) + ID(l)$ can be used as a link ID. In a node-based scheduler such as Orchestra, the corresponding node ID (i.e., $ID(k)$ or $ID(l)$) is hashed instead of the link ID.

Then, $z_{pr}(k, l, ASFN)$ can have a value between 0 and $N_z - 1$. An additional slot's zone is decided by using its slot ID s_{id} . Specifically, we use the array *SHIFT* to allocate slots efficiently in terms of queueing delay. For example, *SHIFT* = [0, 1] when $N_z = 2$ and *SHIFT* = [0, 2, 1, 3] when $N_z = 4$. An additionally allocated slot's zone is shifted from the primary zone by *SHIFT*[s_{id}] as follows:

$$\begin{aligned} z(k, l, ASFN, s_{id}) \\ = mod(z_{pr}(k, l, ASFN) + SHIFT[s_{id}], N_z). \end{aligned} \quad (6)$$

Once a zone is decided, the underlying autonomous scheduler (e.g. Orchestra or ALICE) decides the slot's time offset and channel offset in the corresponding zone.

Lastly, it is important to note that N_z (the number of zones) is the maximum number of slots that a directional link can (potentially) have in a slotframe; it does not mean that a link always uses all the N_z slots. For example, a link continuously monitors its traffic load and utilizes only a single slot in the primary zone when its traffic load is low (i.e., the other $N_z - 1$ slots are deactivated). A³ tries to avoid both under- and over-provisioning.

5.6 Implementation

We implemented the proposed adaptive slot allocation module on Contiki 3.0 and make the source code available to the public.⁵ Note that A³ is orthogonal to underlying autonomous schedulers. We implemented the proposed scheme on top of ALICE and Sender-based Orchestra. We simply extended the existing autonomous schedulers (ALICE and Orchestra) to have the proposed modules by implementing the following functions: (1) Each node updates its own transmission attempt rate at the end of each tx slot. It also estimates each of its neighbor's transmission attempt rate at the end of each rx slot. (2) At the end of a slotframe, each node decides the number of tx/rx slots needed for each of its directional links for the next slotframe. (3) Each slot's zone is selected before a default autonomous scheduler, such as ALICE or Orchestra, determines the slot's time and channel offsets.

We also implemented the additional function to avoid the receiver's false-negative estimation case. To this end, when a sender updates its own transmission attempt rate a , the value a is strictly limited by the value $2 \cdot a_{succ}$ to avoid the case where the transmitter allocates more tx slots than the number of rx slots the receiver allocates. Here, a_{succ} is the transmission success rate (the sender successfully transmitted the packet and received the corresponding acknowledgment from the receiver), thereby, the relation between a and a_{succ} is as follows: $a = a_{succ} + a_{fail}$, where a_{fail} is the transmission failure rate (the sender failed to receive acknowledgment from the receiver). When the transmission rate a exceeds $2 \cdot a_{succ}$, it means that the collision probability of the corresponding link exceeds 50%. This means that the receiver might not be scheduled to listen when the sender transmits a packet or the channel is too bad to be used for communication. To avoid useless slot allocation, when updating a , the value is strictly limited by $2 \cdot a_{succ}$.

6 EVALUATION

We implemented A³ on Contiki [5][25] and evaluate its performance. We use the same experimental environments described in section 4 and used the large public open testbed (FIT IoT-LAB [2][28]) creating multi-hop network of 62 nodes.

We use the four best 802.15.4 channels (15, 20, 25, 26) for TSCH. The number of channel offsets for the unicast slotframe is 4. The slotframe lengths of EB and Broadcast/Default slotframes are 397 and 17, respectively. The duration of the time slot is 10 ms. We used RPL [34] storing mode for routing and MRHOF [11] with ETX for the objective function. We enabled DAO-ACK option to generate reliable downstream paths. We use 62 M3 nodes (MCU: ARM Cortex M3, 32-bits, 72 Mhz, 64kB RAM, Radio communication: 802.15.4 PHY standard, 2.4 Ghz). We set the transmission power as -17 dBm

⁵URL: <https://github.com/skimskim/A3>.

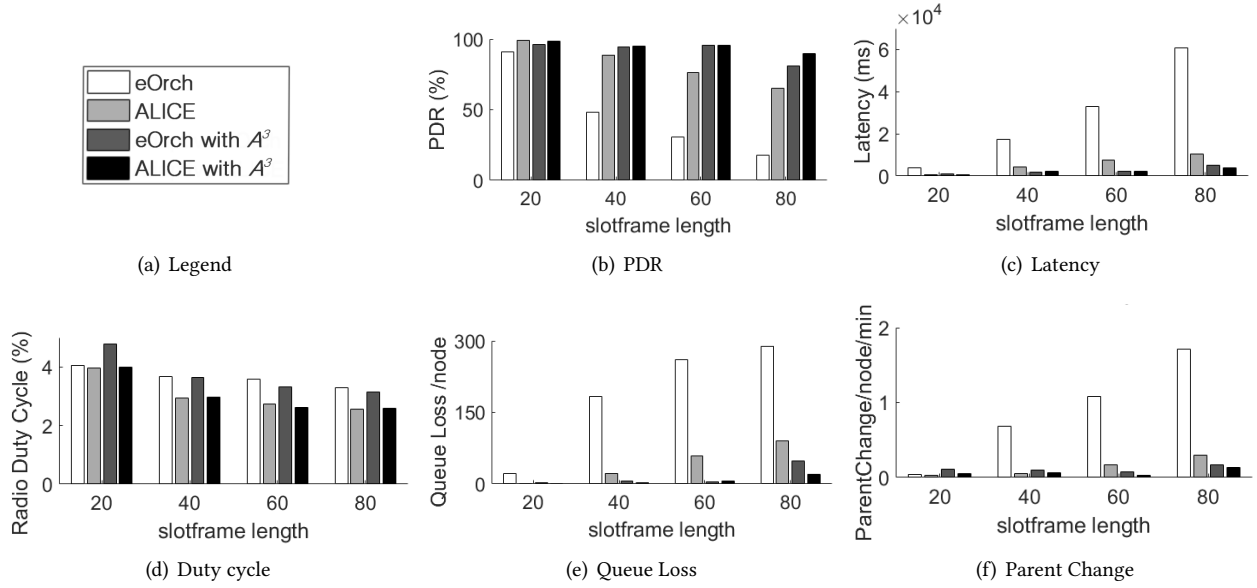


Figure 7: Performance by the slotframe length in a dynamic topology (traffic load: 4pkts/min)

thereby creating a 6-hop network. We set N_z of the A^3 as 4 unless otherwise noted, and thereby each directional link can have up to 4 tx and 4 rx slots in a slotframe.

6.1 Impact of Unicast Slotframe Length (Dynamic Topology)

We increased the slotframe length from 20 to 80 at 4 pkts/min traffic load. As in Section 4, it means that each packet interval is set randomly while making each node send 4 packets per minute to the root and the root sends 4 packets per minute to each node *on average*. Routing topology is dynamically changed as RPL changes the parent nodes. Although each node generates the same amount of traffic, it can experience a dynamically changing traffic load as the routing topology changes since its *forwarding burden* depends on the sub-network size. We compare the performance of the existing autonomous schedulers *with and without* the proposed adaptive autonomous slot allocation module A^3 .

Figure 7 shows the evaluation results by the slotframe length. Without A^3 , both eOrchestra and ALICE had to assign only a single tx/rx slot per slotframe for a directional link, regardless of the used slotframe length. Both schemes show fairly good performance when the slotframe length is 20, meaning that providing a tx/rx slot for a link per 20 slots is enough to handle the given traffic load. As slotframe length increases, however, they show poor performance, as we have seen in Section 4.

In contrast, with A^3 , both schedulers improve their performance significantly in all aspects (PDR, latency, duty cycle, queue loss, and topology stability). Recall that each node of a multi-hop network has a different traffic load according to

routing topology. When slotframe length is 80, for example, A^3 enables the nodes experiencing heavy traffic load (those near the root) to utilize up to 4 tx/rx slots per slotframe, as if the slotframe length is 20. Thereby, A^3 reduces latency (figure 7(c)) and queue loss (figure 7(e)). This results in high PDR even with a long slotframe length (figure 7(b)). For leaf-side nodes that experience low traffic load, A^3 allocates only a single tx/rx slot per slotframe just as the default schedulers, which minimizes energy wastage.

Figure 7(d) shows that using A^3 maintains or increases duty cycle for a short slotframe since it can over-provision slots occasionally. Interestingly, however, although A^3 utilizes more slots, it reduces radio duty cycle for a long slotframe (a congested scenario). This is because, by utilizing more slots, A^3 maintains routing topology more stably (figure 7(f)). Without A^3 , both Orchestra and ALICE suffer packet collisions which change routing topology changes and/or prevent time synchronization. Therefore, they end up generating more control packets to repair routing topology. It verifies that although A^3 is not mainly designed to reduce energy consumption, it can actually achieve a lower duty cycle when traffic load is high due to its robustness.

6.2 Impact of Traffic Load (Dynamic Topology)

We also evaluate the performance with varying traffic loads. In this experiment, we use different slotframe lengths for the two autonomous schedulers. Since ALICE performs better than eOrchestra at the same slotframe length, we assign the slotframe length 20 for eOrchestra and 40 for ALICE, respectively. We increased the traffic load from 4 pkts/min to

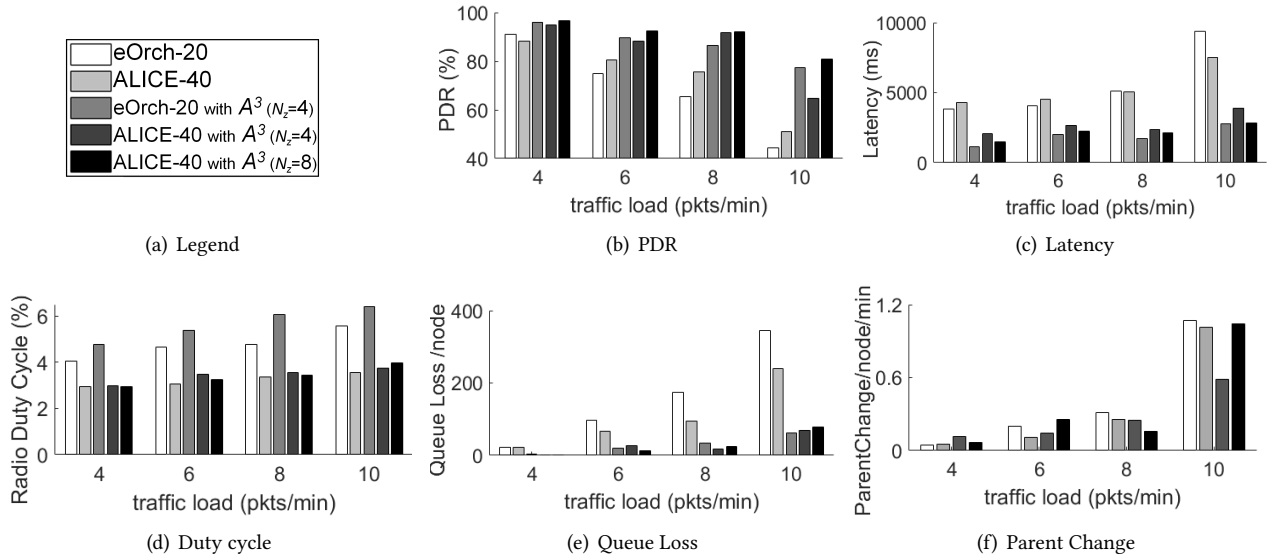


Figure 8: Performance by the traffic load in a dynamic topology

10 pkts/min. At k pkts/min traffic load, the root node sends k pkts/min to each node and each node sends k pkts/min to the root on average. In this section, to show the effect of the number of zones, we use 4 and 8 for N_z of A³.

The results are described in figure 8. At 4 pkts/min traffic load, both eOrchestra and ALICE (without A³) achieve high PDR. However, as the traffic load increases, both schemes could not handle it. Especially the root-side nodes that should forward a lot of packets between the root and its sub-network excessively suffer from over-loaded traffic. The transmission queue increases and the packets suffer from long queueing delay and are dropped from the queue (figure 8(e)). As a result, PDR significantly decreases, and they fail in providing reliable communication service (figure 8(b)). Due to the heavy traffic loads, the network loses important control packets, and RPL and TSCH become unstable. Some nodes lose the TSCH connection and try to join the TSCH network again, scanning the channels continuously, which is the main reason for the high radio duty cycle at the high traffic load (figure 8(d)).

The root cause of the dismal performance is insufficient communication opportunities assigned for each scheme. The proposed adaptive resource allocation module A³ enables *dynamic* and *flexible* allocation of additional slots according to the real-time traffic loads. Utilizing more slots, both schemes with A³ build a more stable and reliable network under heavy traffic loads. Consequently, A³ achieves much higher PDR and much lower latency at the cost of negligible amounts of the radio duty cycle. In addition, we found that even when traffic load is high, only a few nodes that have a large number of descendant nodes utilize additional slots. It turns out that

allocating additional slots to *just a few nodes* can significantly improve overall network performance. Compared to figure 7, figure 8 includes another case of A³ with $N_z = 8$, showing that it is more robust to heavy traffic than using $N_z = 4$ since it can allocate more slots per directional link.

The results also show A³'s limitations. When traffic load becomes too high, packet collision increases even when A³ allocates more slots. This is verified by figure 8(f), which shows that A³ experiences more RPL parent changes as traffic load increases. This is because receivers observe more collision and unavailable cases from their rx slots, making the estimated traffic load ($\hat{a}_{i,rx}$) slowly updated and resulting in more over-provisioning. As a result, unlike figure 7(d), figure 8(d) shows that A³'s duty cycle is not always lower than the default schedulers.

6.3 Impact of Dynamic Traffic Load (Fixed Topology)

To take a deeper look, we perform another experiment with ALICE-A³ that shows how A³ reacts when a directional link experiences sudden traffic load changes. To focus solely on traffic load changes, we fixed the topology as shown in figure 9, which is a snapshot of routing topology formed by RPL. The experiment is 30-minute long and each node sends and receives 2 pkts/min to and from the root, respectively. In addition, there are 21 particular nodes marked with thick red circles in figure 9, which are node A's descendants. These nodes generate different upstream traffic load over time. Each of the 21 nodes generates upstream traffic of 2 pkts/min for the first 8 minutes, 4 pkts/min for the next 8 minutes, 10 pkts/min for the next 8 minutes, and 2 pkts/min for the last

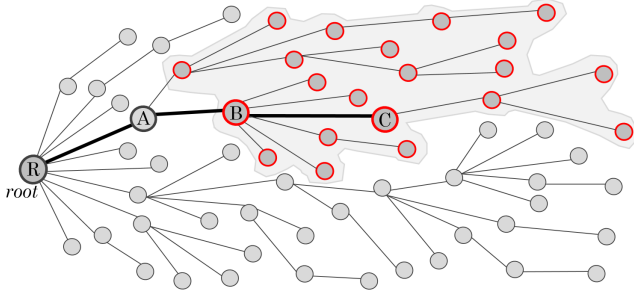


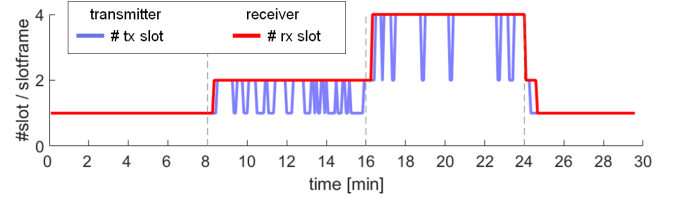
Figure 9: RPL topology used for the fixed-topology experiments in Sections 6.3, 6.4, and 6.5

6 minutes. Note again that all the traffic loads are average loads and each packet interval is randomly determined.

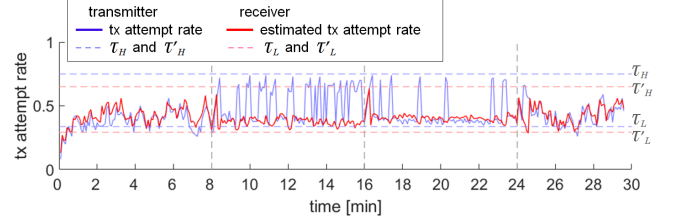
Figures 10(a), 10(c), and 10(e) show the number of tx/rx slots scheduled for the directional links (A, R), (B, A), and (C, B) over time. The number of tx (or rx) slots allocated for a slotframe over time is described with thick blue (or red) lines. Figures 10(b), 10(d), and 10(f) show A^3 's transmission attempt ratio for the directional links (A, R), (B, A), and (C, B) over time. The average (EWMA) transmission attempt rate calculated at the sender (or the receiver) is described as blue (or red) lines, respectively. By considering the observed/estimated transmission attempt rates and the increase/decrease thresholds for the number of slots, A^3 decides the number of tx/rx slots per slotframe (between 1 and N_z).

At around 8 minute, 21 nodes increase their traffic load from 2 pkts/min to 4 pkts/min. At this time, the links (C, B) and (B, A) do not increase the number of slots (figures 10(c) and 10(e)). Since the nodes B and C have only 10 and 3 nodes in their sub-tree, respectively, transmission attempt rates of both links are below the increase threshold (figures 10(d) and 10(f)). However, given that node A has all the 21 nodes in its sub-tree, transmission attempt rate of the link (A, R) crossed the increase thresholds, both at the sender A and the receiver R (figure 10(b)). Thus, the number of tx/rx slots for the link (A, R) is doubled (1 to 2) within 20 seconds. The 20-second transition period is needed for the traffic load change is detected by the EWMA filter. It is important to note that the number of rx slots is increased slightly earlier than that of tx slots since $\tau'_H < \tau_H$, resulting in a safe transition without control packet exchanges. After the number of slots increases, the transmission attempt rate is halved since it is distributed over the added slots.

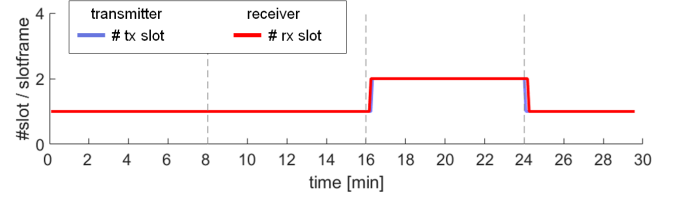
At around 16 minutes after the experiment starts, the traffic load increased from 4 pkts/min to 10 pkts/min, and at this time, the transmission attempt rates of the two links (A, R) and (B, A) hit the increase threshold (τ_H or τ'_H) as shown in figures 10(b) and 10(d). Thereby, both links double the number of tx/rx slots per slotframe within 20 seconds and 15



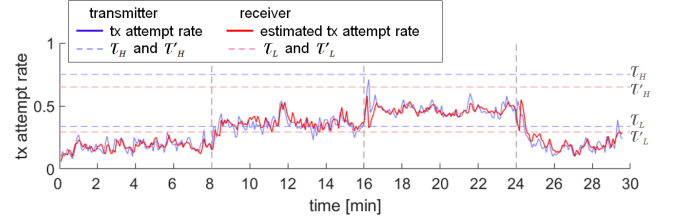
(a) The number of allocated slots for link (A, R) over time



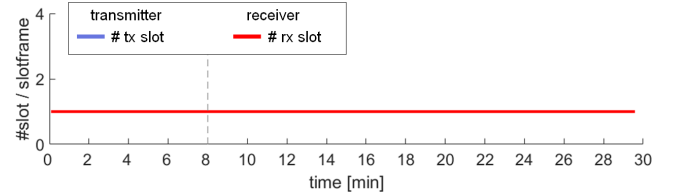
(b) Transmission attempt rate at link (A, R) over time



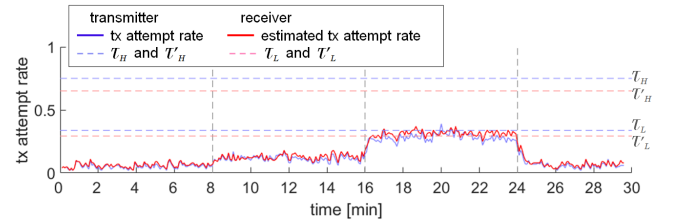
(c) The number of allocated slots for link (B, A) over time



(d) Transmission attempt rate at link (B, A) over time



(e) The number of allocated slots for link (C, B) over time



(f) Transmission attempt rate at link (C, B) over time

Figure 10: A^3 's slot allocation by traffic load over time

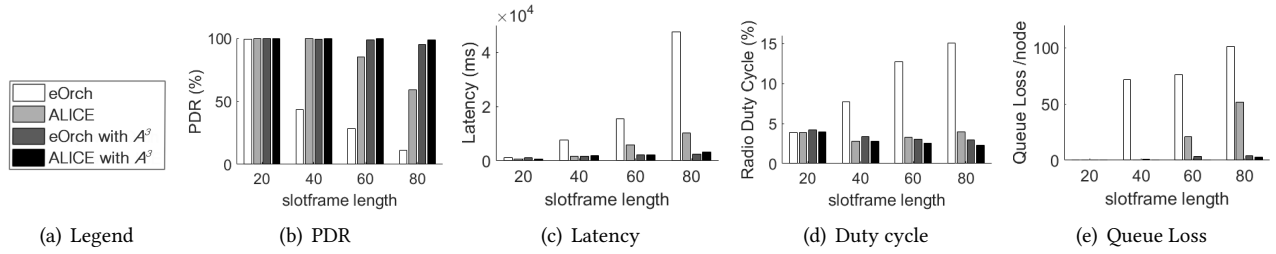


Figure 11: Performance by the slotframe length in the fixed topology (traffic load: 4pkts/min)

seconds, respectively. Again, A³ achieves safe transition for both links by increasing the number of rx slots earlier than that of tx slots. After doubling the number of slots per slotframe, the transmission attempt ratio of each slot becomes halved due to the distributed traffic load over increased slots. On the other hand, the link (C, B) does not increase the number of slots per slotframe since it does not experience much traffic due to its small sub-network size (figures 10(e) and 10(f)). It is clearly shown that although the number of zones is 4, the number of tx/rx slots that are actually allocated for the link (C, B) is always 1.

At around 24 minutes, the 21 nodes significantly decrease their upstream traffic load from 10 pkts/min to 2 pkts/min. Thereby, the transmission attempt rates of the links (A, R) and (B, A) go below the decrease threshold (τ_L or τ'_L) as shown in figures 10(b) and 10(d), and both links halve the number of tx/rx slots within 5 seconds and 15 seconds, respectively (10(a) and 10(c)). Since the link (A, R) experiences more dramatic traffic load reduction than the link (B, A), it decreases tx/rx slots much faster. Note that unlike when increasing the number of slots, the number of rx slots decreases slightly later than that of tx slots since $\tau'_L < \tau_L$. A³ achieves safe transition again in an autonomous manner due to the careful threshold design. After decreasing the number of slots, traffic is concentrated on the reduced number of slots, doubling the transmission attempt rates. The link (A, R) experiences low traffic even after decreasing the number of slots once, decreasing it again 35 seconds later.

Overall, the results show that A³ well adapts to the varying traffic load and allocates the adequate number of slots based on the real-time traffic load. In this experiment, the achieved upstream/downstream PDR is 99.89%/ 99.92%, the upstream/downstream latency is 1,625 ms/1,351 ms, and the average radio duty cycle is 2.66%.

6.4 Impact of Unicast Slotframe Length (Fixed Topology)

We repeat the same experiment that we did in section 6.1 and 6.2 over the fixed routing topology used in the section 6.3. By fixing the network topology, we can remove the unnecessary variations from the routing layer and focus on

the effect of link-layer scheduling. When there are too many packets on the network compared to the provided communication opportunity, each slot becomes extremely crowded, resulting in excessive packet losses. Without a parent change, traffic congestion degrades performance more seriously. Important control packets are dropped, which can ruin TSCH time synchronization. In this case, nodes consume significant amounts of energy, continuously scanning the channels to find a valid TSCH time source again.

As in section 6.1, we evaluate the performance while varying the length of slotframe from 20 to 80. The results are described in figure 11. Compared to eOrchestra, ALICE shows better performance. In case of eOrchestra, some leaf-side nodes lost TSCH connection and consume much energy (figure 11(d)) for scanning a TSCH network. Applying A³ on top of the existing autonomous schedulers, both ALICE and eOrchestra opportunistically utilize more slots in a slotframe according to each directional link's traffic load. The results in figure 11 show that performance gain by using A³ in a fixed topology is much more significant than that in dynamic topology (figure 7 in section 6.1). This implies that RPL's parent change helps to partly avoid traffic congestion. In addition, we can infer that when a node has only one parent candidate (a case of sparse topology), adaptive slot scheduling is necessary to handle traffic congestion.

6.5 Impact of Traffic Load (Fixed Topology)

We also repeated the same experiment in section 6.2 (performance with varying traffic load) in the fixed topology. We assign slotframe length 20 for eOrchestra and 40 for ALICE, respectively, since ALICE performs better than eOrchestra at the same slotframe length. At this time, we added a heavier traffic load: 15 pkts/min.

The results are described in figure 12. Without A³, both eOrchestra and ALICE provide a fixed number of slots in a slotframe regardless of the traffic load. Due to lack of routing capability, traffic congestion directly impacts performance, resulting in worse performance compared to the results in figure 8. However, equipped with A³, each link improves its communication performance significantly by allocating additional slots. Utilizing more communication slots, the

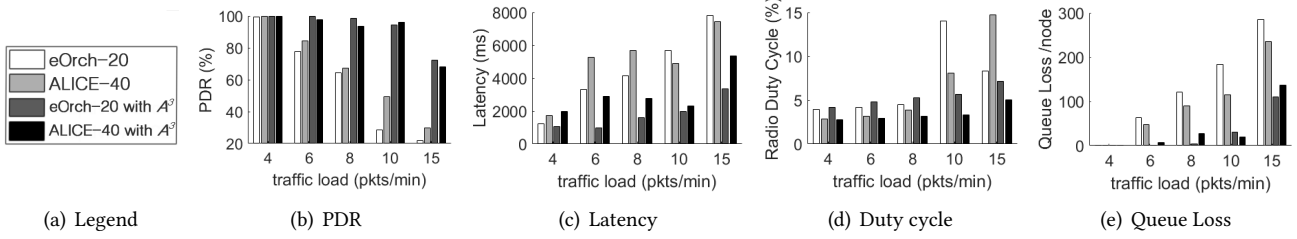


Figure 12: Performance by the traffic load in the fixed topology

links with high traffic loads assign more communication slots and clean the transmission queue quickly. Consequently, they decrease the number of queue losses significantly (figure 12(e)), reducing latency (figure 12(c)) and increasing PDR (figure 12(b)).

Interestingly, in this set of experiments, A^3 also improves duty-cycle performance significantly (figure 12(d)). Although A^3 can slightly over-provision slots sometimes and consume more energy unnecessarily, not using A^3 is much more harmful in a fixed topology. This is because in the fixed topology, traffic congestion incurs continuous packet losses in a directional link, which easily makes a node lose time synchronization.

In summary, we have seen that A^3 could be used with any autonomous TSCH scheduler, such as Orchestra and ALICE, and enables the existing autonomous schedulers to enjoy the enhancements in all performance metrics: higher PDR, lower latency, and (sometimes) lower duty cycle. A^3 traffic load estimation and its careful parameter choices provide safe transition of the number of slots without control packet exchanges when traffic load changes.

7 DISCUSSION

Recently, there have been two pieces of work on adaptive distributed slot scheduling for TSCH, TESLA [16] and OST [15], that seem promising. Both schemes are evaluated on a real testbed network and show significant performance improvement over Orchestra and/or ALICE. Both schemes require explicit control packet exchanges for adaptive scheduling but adopt novel techniques for robust operation and low control overhead.

Unlike A^3 , however, TESLA design is tightly coupled with receiver-based Orchestra and cannot be applied to various TSCH schedulers, such as ALICE. This means that TESLA's performance is fundamentally bounded by the node-based scheduling concept of Orchestra. It has also been shown that receiver-based Orchestra provides worse performance than sender-based Orchestra.

On the other hand, OST works for directional links and uses both the shared link and dedicated link options. Given that OST outperforms TESLA and implementation codes of

both OST and A^3 are open to public, we see that a performance evaluation study of A^3 and OST would be valuable for the community. It has been more than 5 years since Orchestra shedded light on TSCH scheduling by using an autonomous approach, and the community now has the two adaptive schedulers, one (OST) in a distributed manner and the other (A^3) in an autonomous manner. In terms of robust operation, there might be an interesting trade-off between control packet loss (distributed approach) and estimation error (autonomous approach). What is the right way to go forward?

In a broader view, as LLN technology is becoming fairly mature, we need to consider more about its applications. LLN hardware has become much more powerful and system architectures have evolved accordingly [4, 10, 13, 23]. LLNs can now deliver much heavier traffic due to various run-time adaptation techniques, not only resource scheduling but also load balancing [9, 19] and transmission power control [8, 20]. Full-scale TCP is not a luxury anymore [17, 18, 22], bringing real "I" to "I"oT. We see that assembling all these pieces together in a synergistic manner and finding its value in practical applications would be an important next step.

8 CONCLUSION

The current autonomous TSCH schedulers provide light and simple slotframe scheduling methods, not generating any packet exchange overhead. However, they just allocate the fixed number of tx/rx slots per slotframe based on the number of neighbors regardless of each link's traffic load. In this work, we have pointed out that the existing autonomous schedulers cannot provide reliable service due to lack of adaptability. As far as we know, this is the first work that proposes a *fully autonomous* and *real-time traffic load* -based *adaptive* slot allocation scheme for TSCH scheduling. As an independent module, A^3 can be applied to any autonomous TSCH scheduler such as Orchestra and ALICE. Through the extensive evaluation on a real-world and large public testbed, we verified that A^3 provides robust performance even in dynamic and heavy traffic load scenarios. Experimental results show that A^3 achieves more than six times better PDR and ten times less latency.

APPENDIX

A. Proof of Lemma 1: Consider an EWMA formula

$$Z_i = \lambda \cdot Y_i + (1 - \lambda) \cdot Z_{i-1}.$$

The expectation of Z_i , $E[Z_i]$, is

$$(1 - (1 - \lambda)^k) \cdot p_c + (1 - \lambda)^k \cdot p_o,$$

where $p_o = E[Z_{i-k}]$. Using a bound that $\text{Var}[Y_j] \leq 1/4$, we derive an upper bound for the variance of Z_i as

$$\text{Var}[Z_i] \leq \frac{\lambda(1 - (1 - \lambda)^{2i})}{2 - \lambda} \cdot \frac{1}{4} \leq \frac{\lambda}{4 \cdot (2 - \lambda)}.$$

The probability of false-negative, $\Pr(Z_i \leq \tau'_H)$, is the probability that the estimation Z_i is less than or equal to the receiver's threshold τ'_H .

$$\Pr(Z_i \leq \tau'_H) = \Pr(Z_i - E[Z_i] \leq \tau'_H - E[Z_i])$$

Applying the Chebyshev's inequality, we obtain

$$\Pr(Z_i - E[Z_i] \leq -t \cdot \text{Var}[Z_i]) \leq \frac{1}{1 + t^2}.$$

Now, we rewrite $\Pr(Z_i - E[Z_i] \leq \tau'_H - E[Z_i])$ as

$$\Pr(Z_i - E[Z_i] \leq \text{Var}[Z_i] \cdot \frac{\tau'_H - E[Z_i]}{\text{Var}[Z_i]})$$

and applying the above result to obtain an upper bound as follows.

$$\begin{aligned} \Pr(Z_i \leq \delta'_H) &\leq \frac{1}{1 + (\frac{E[Z_i] - \tau'_H}{\text{Var}[Z_i]})^2} \\ &\leq \frac{1}{1 + (\frac{(1 - (1 - \lambda)^k) \cdot p_c + (1 - \lambda)^k \cdot p_o - \tau'_H}{\frac{\lambda}{4(2 - \lambda)}})^2} \end{aligned}$$

Above upper bound is further simplified to

$$\frac{\lambda^2}{\lambda^2 + (4(2 - \lambda)(p_c - (1 - \lambda)^k(p_c - p_o) - \tau'_H))^2}$$

which is minimized to

$$\frac{\lambda^2}{\lambda^2 + (4(2 - \lambda)(\Delta_H - (1 - \lambda)^k(\tau_H - p_o)))^2}$$

at $p_c = \tau_H$.

B. Proof of Lemma 2: The probability of false-positive is $\Pr(Z_i \geq \tau'_H)$, represented as below.

$$\begin{aligned} \Pr(Z_i \geq \tau'_H) &= \Pr(Z_i - E[Z_i] \geq \tau'_H - E[Z_i]) \\ &= \Pr(Z_i - E[Z_i] \geq \text{Var}[Z_i] \cdot \frac{\tau'_H - E[Z_i]}{\text{Var}[Z_i]}) \\ &\leq \frac{1}{1 + (\frac{E[Z_i] - \tau'_H}{\text{Var}[Z_i]})^2} \\ &\leq \frac{1}{1 + (\frac{\tau'_H - (1 - (1 - \lambda)^k) \cdot p_c - (1 - \lambda)^k \cdot p_o}{\frac{\lambda}{4(2 - \lambda)}})^2} \\ &= \frac{\lambda^2}{\lambda^2 + (4(2 - \lambda)(\tau'_H - (1 - \lambda)^k(p_c - p_o)))^2} \end{aligned}$$

ACKNOWLEDGMENTS

This work was partly supported by the ICT at Seoul National University (SNU), partly by the Institute for Industrial Systems Innovation of SNU, partly by BK21 FOUR Intelligence Computing (Dept. of Computer Science and Engineering, SNU) funded by NRF of Korea (4199990214639), partly by NRF of Korea Grant funded by the Korean Government (MSIP) (NRF-2016R1A5A1012966), and partly by NRF grant funded by the Korea government (MSIT) (No. 2020R1A2C2101815).

REFERENCES

- [1] N. Accettura, M. Palattella, G. Boggia, L. Grieco, and M. Dohler. 2013. DeTAS: A decentralized traffic aware scheduling technique enabling IoT-compliant multi-hop low-power and lossy networks. *Second IEEE WoWMoM Workshop on IoT-SoS*, 337–350.
- [2] C. Adjih, E. Baccelli, E. Fleury, G. Harter, N. Mitton, T. Noel, R. Gibollet, F. Marcel, G. Schreiner, J. Vandaele, and T. Watteyne. 2015. FIT IoT-LAB: A large scale open experimental IoT testbed. *IEEE World Forum on Internet of Things (WF-IoT)* (2015).
- [3] ANSI/ISA. 2011. ANSI/ISA-100.11a-2011 Wireless systems for industrial automation: Process control and related applications. *ISA* (2011).
- [4] Holly Chiang, Hudson Ayers, Daniel Giffin, Amit Levy, and Philip Levis. [n. d.]. Power Clocks: Dynamic Multi-Clock Management for Embedded Systems. *Energy* 1 ([n. d.]), 2.
- [5] A. Dunkels, B. Gronvall, and T. Voigt. 2004. Contiki - a lightweight and flexible operating system for tiny networked sensors. In *IEEE LCN*. 455–462.
- [6] S. Duquenooy, B. Nahas, O. Landsiedel, and T. Watteyne. 2015. Orchestra: Robust mesh networks through autonomously scheduled TSCH. *ACM SenSys* (2015).
- [7] Atis Elsts, Seohyang Kim, Hyung-Sin Kim, and Chongkwon Kim. 2020. An empirical survey of autonomous scheduling methods for TSCH. *IEEE Access* 8 (2020), 67147–67165.
- [8] Hyung-Sin Kim et al. 2017. Do not lose bandwidth: Adaptive transmission power and multihop topology control. In *IEEE DCOSS*. 99–108.
- [9] H.-S. Kim et al. 2017. Load balancing under heavy traffic in RPL routing protocol for low power and lossy networks. *IEEE Transactions on Mobile Computing* 16, 4 (2017), 964–979.

- [10] Hyung-Sin Kim et al. 2018. System architecture directions for post-soc/32-bit networked sensors. In *ACM SenSys*. 264–277.
- [11] O. Gnawali and P. Levis. September 2012. The Minimum Rank with Hysteresis Objective Function. *IETF, RFC 6719* (September 2012).
- [12] IEC. April 27, 2010. Industrial Communication Networks Wireless Communication Network and Communication Profiles WirelessHART. *IEC 62591 Ed. 1.0 b:2010* (April 27, 2010).
- [13] Neal Jackson, Joshua Adkins, and Prabal Dutta. 2019. Capacity over capacitance for reliable energy harvesting sensors. In *Proceedings of the 18th International Conference on Information Processing in Sensor Networks*. 193–204.
- [14] K. Jain, J. Padhye, V. Padmanabhan, and L. Qiu. 2003. Impact of Interference on Multi-hop Wireless Network Performance. *ACM Mobicom* (2003).
- [15] S. Jeong, H. Kim, J. Paek, and S. Bahk. 2020. OST: On-Demand TSCH Scheduling with Traffic-awareness. *IEEE INFOCOM* (2020).
- [16] S. Jeong, J. Paek, H. Kim, and S. Bahk. September 2019. TESLA: Traffic-Aware Elastic Slotframe Adjustment in TSCH Networks. *IEEE ACCESS* (September 2019).
- [17] Hyung-Sin Kim, Heesu Im, Myung-Sup Lee, Jeongyeup Paek, and Saewoong Bahk. 2015. A measurement study of TCP over RPL in low-power and lossy networks. *Journal of Communications and Networks* 17, 6 (2015), 647–655.
- [18] Hyung-Sin Kim, Sam Kumar, and David E Culler. 2019. Thread/OpenThread: A compromise in low-power wireless multihop network architecture for the Internet of Things. *IEEE Communications Magazine* 57, 7 (2019), 55–61.
- [19] Hyung-Sin Kim, Jeongyeup Paek, and Saewoong Bahk. 2015. QU-RPL: Queue utilization based RPL for load balancing in large scale industrial applications. In *2015 12th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. IEEE, 265–273.
- [20] Hyung-Sin Kim, Jeongyeup Paek, David E Culler, and Saewoong Bahk. 2020. PC-RPL: Joint control of routing topology and transmission power in real low-power and lossy networks. *ACM Transactions on Sensor Networks (TOSN)* 16, 2 (2020), 1–32.
- [21] S. Kim, H. Kim, and C. Kim. April 16–18, 2019. ALICE: Autonomous Link-based Cell Scheduling for TSCH. In *The 18th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN 2019)* (April 16–18, 2019).
- [22] Sam Kumar, Michael P Andersen, Hyung-Sin Kim, and David E Culler. 2020. Performant TCP for Low-Power Wireless Networks. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2020)*. 911–932.
- [23] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. 2017. Multiprogramming a 64kb computer safely and efficiently. In *Proceedings of the 26th Symposium on Operating Systems Principles*. 234–251.
- [24] A. Morell, X. Vilajosana, J. Vicario, and T. Watteyne. May 2013. Label switching over IEEE 802.15.4e networks. In *Transactions on Emerging Telecommunications Technologies*, Vol. 24. 458–475.
- [25] Contiki OS. Open. URL. <http://www.contiki-os.org>. (Open).
- [26] M. Palattella, N. Accettura, M. Dohler, L. Grieco, and G. Boggia. 2012. Traffic Aware Scheduling Algorithm for reliable low-power multi-hop IEEE 802.15. 4e networks. *IEEE PIMRC* (2012).
- [27] M. Palattella, T. Watteyne, Q. Wang, K. Muraoka, N. Accettura, D. Dujovne, L. Grieco, and T. Engel. 2016. On-the-fly bandwidth reservation for 6tisch wireless industrial networks. *IEEE Sensors* 16, 2 (2016), 555–560.
- [28] FIT/IoT-LAB public testbed. Open. URL. <http://www.iot-lab.info>. (Open).
- [29] IEEE Computer Society. 1 October 2003. IEEE Computer Society, 2003, IEEE Standards 802.15.4, IEEE Standard for Information technology, Part 15.4: Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (LR-WPANs),. *IEEE* (1 October 2003).
- [30] IEEE Computer Society. 2012. IEEE standard for information technology, 802.15.4e, Part. 15.4: Low-Rate Wireless Personal Area Networks (LR-WPANs) Amendment 1: MAC sublayer. *IEEE Computer Society* (2012).
- [31] R. Soua, P. Minet, and E. Livolant. 2012. MODESA: an optimized multichannel slot assignment for raw data convergecast in wireless sensor networks. *IEEE IPCCC* (2012).
- [32] R. Soua, P. Minet, and E. Livolant. 2015. Wave: a distributed scheduling algorithm for convergecast in IEEE 802.15.4e TSCH networks. *Transactions on Emerging Telecommunications Technologies* 27, 4 (2015), 557–575.
- [33] R. Tavakoli, M. Nabi, T. Basten, and K. Goossens. 2018. Topology Management and TSCH Scheduling for Low-Latency Convergecast in In-Vehicle WSNs. *IEEE Transactions on Industrial Informatics* (2018).
- [34] P. Thubert. March 2012. Objective Function Zero for the Routing Protocol for Low-Power and Lossy Networks (RPL). *IETF, RFC 6552* (March 2012).
- [35] Q. Wang, X. Vilajosana, and T. Watteyne. November 2018. 6TiSCH Operation Sublayer (6top) Protocol (6P). *RFC 8480, 6TiSCH, IETF* (November 2018).
- [36] IETF 6TiSCH WG. Open. URL. <https://datatracker.ietf.org/group/6tisch/about/>. (Open).
- [37] P. Zand, A. Dilo, and P. Havinga. June 2013. D-MSR: A Distributed Network Management Scheme for Real-Time Monitoring and Process Control Applications in Wireless Industrial Automation. *Sensors* (June 2013).